

Revon: Git-Inspired Merkle Hash Trie Versioning for Structured Data

Atharva Sheersh Pandey¹, Adhyan Jain¹, Poornima Nedunchezian²

atharva.sheersh2024@vitstudent.ac.in; adhyan.jain2024@vitstudent.ac.in;

poornima.n@vit.ac.in

School of Computer Science and Engineering

VIT Vellore

Abstract: Versioning structured data means keeping exact historical states, handling sparse updates, and comparing versions without copying the full dataset each time. Revon is a Python prototype built on an immutable, fixed-depth Merkle hash trie. Its hybrid variant, Revon-H, stores addressed changesets and chooses between log aggregation and hash-pruned differencing. We tested Snapshot, Log-only, forced-Merkle Revon-M, Revon-H, and Dolt 2.3.1 on synthetic workloads of up to 100,000 rows, across 540 successful executions. In that synthetic study, median paired Dolt/Revon-H diff-latency ratios ranged from 7.14 to 625.10, and all seven-trial 95% bootstrap intervals favored Revon-H latency. Those are interface-level results: the comparison includes different APIs and does not isolate tree or index performance. A separate three-seed study could not resolve the 50-commit ablation result. A separate public TLC record evaluation reaches 1,000,000 records with generated update histories. We used one Windows host; the earlier synthetic study also includes WSL2 on that computer. The results show workload-dependent trade-offs for this prototype; they do not establish general or production-level superiority.

1 INTRODUCTION

Structured-data version control has to do more than archive files. It must preserve exact versions, apply sparse updates without rebuilding the whole state, restore earlier versions, and compare states efficiently. Git offers a useful starting point: immutable objects named by their content, linked through parent commits (Chacon and Straub, 2014). Its objects are organized around files and directories, however. Record stores have a different shape: keys may remain stable while update density changes, and two

versions may differ by a few edits or by a long commit history.

Revon asks whether a fixed-depth Merkle hash trie can combine incremental structural sharing with an operation index. The main variant, Revon-H, uses the log path for short or sparse intervals and switches to Merkle comparison after a calibration threshold fixed before evaluation. SQLite supplies atomic, durable storage; it does not define Revon’s versioning algorithm.

We use Dolt as the production comparator because it combines SQL, a Git-style commit graph, and content-addressed Prolly Trees (DoltHub, 2024), (Sehn, 2024). Revon is a smaller research prototype, not a feature-for-feature replacement for Dolt. Our results compare Revon’s in-process Python API with a Dolt CLI workflow; they do not isolate the performance of either tree structure.

1.1 Research Questions

- RQ1: How do Revon-H’s latency and storage compare with Snapshot, Log-only, and Dolt across dataset sizes, mutation densities, and update localities?
- RQ2: Does choosing between log and Merkle diff improve latency over forced-Merkle Revon-M while preserving correctness?
- RQ3: How do branching factor and trie depth affect latency, storage, and the amount of work examined?

1.2 Contributions

- We implement a canonical SHA-256-addressed, fixed-depth trie with incremental rewriting of paths affected by multi-key updates.
- We design a hybrid diff policy that combines addressed changesets with hash-pruned tree comparison.

- We build an atomic SQLite repository for objects, commits, aliases, measurements, and HEAD, and verify object hashes after reopening it.
- We compare five systems on shared deterministic workloads, using common diff and checkout outputs. We report Dolt’s two initial-import workflows separately, randomize and balance execution order, sample process-tree RSS externally, retain raw evidence, and audit its integrity.
- We measure how branching factor and depth affect the fixed trie, rather than assuming that $b=8$ and $d=4$ are universally best.

2 RELATED WORK

2.1 Content Addressing and Persistent Structures

Merkle’s authenticated-tree construction showed that a compact root digest can commit to a larger collection (Merkle, 1988). Git and IPFS apply content addressing to software history and networked content (Chacon and Straub, 2014), (Benet, 2014). Persistent-data-structure theory formalized path copying and access to earlier states (Driscoll et al., 1989). Later persistent dictionaries explore ordered, disk-oriented alternatives: stratified B-trees (Twigg et al., 2011) and fully persistent B-trees (Brodal et al., 2020). Revon instead routes hashed keys through a fixed-depth trie, trading ordered range access for hash-pruned comparison. Scientific array systems such as SciDB and TimeArr preserve versions using array-specific storage layouts and deltas (Seering et al., 2012), (Soroush and Balazinska, 2013). TardisDB integrates version access with a main-memory database, while Saquib et al. study replicated versioned structures across wide-area systems (Saquib et al., 2023), (Schüle et al., 2019). These systems illuminate adjacent design choices, but they do not compare Revon’s log and Merkle diff paths over the same history.

Merkle Search Trees combine search-tree ordering with Merkle authentication to reconcile state (Auvolat and Taïani, 2019). Revon also prunes comparisons by hash, but uses fixed-depth routing and a centralized linear history rather than a replicated CRDT. LBFS introduced content-defined chunking for redundancy detection (Muthitacharoen et al., 2001); Dolt and Noms use related boundary strategies. Ordered

indexes preserve range operations, while content-defined boundaries can improve deduplication when data shifts. Revon’s current benchmark measures neither capability.

2.2 Structured Dataset Versioning

Dataset-versioning research covers collaboration, storage layout, and retrieval. DataHub studies collaborative dataset history at scale (Bhardwaj et al., 2015). Decibel brings branching into a relational engine and compares version-first, tuple-first, and hybrid layouts (Maddox et al., 2016). OrpheusDB adds versioning to a conventional DBMS and optimizes version retrieval through partitioning (Huang et al., 2017). Bhattacharjee et al. formalize the storage-versus-reconstruction trade-off (Bhattacharjee et al., 2015), while Delta Lake uses transaction logs and checkpoints for ACID tables (Armbrust et al., 2020). More recent work studies how to choose materialized versions and deltas over a version graph, subject to storage and retrieval costs (Guo et al., 2024). A FAIR-compliant proposal focuses on identifying and describing dataset releases consistently (González-Cebrián et al., 2024); it complements storage research but does not provide a diff algorithm. Revon addresses a narrower question with a key/value prototype and does not solve these broader dataset-management problems.

2.3 System Design Gap

Noms stores structured values in a Merkle DAG and supports diff and synchronization (Attic Labs, n.d.). ForkBase combines content addressing with fork and deduplication semantics (Wang et al., 2018). Dolt’s content-addressed Prolly Trees combine ordered B-tree-like indexing, structural sharing, and native diff (DoltHub, 2024), (Sehn, 2024). Git4Data is another nearby direction: it presents SQL-level snapshot, branch, diff, and merge operations over a relational engine (Gou et al., 2026). The available Git4Data manuscript is an arXiv preprint, so we do not treat its results here as peer-reviewed or independently reproduced evidence. Revon has a narrower scope: it is a single-writer, linear-history key/value prototype with fixed-depth hash routing, not a SQL database or branching system.

System or study	Representation or focus	Relevance to Revon
Persistent DS (Driscoll et al., 1989)	Path copying	Formal structural-sharing basis
Git (Chacon and Straub, 2014)	Object DAG	Identity and commit ancestry; file-oriented
IPFS (Benet, 2014)	Merkle DAG	General content-addressed versioning
MST (Auvolat and Taïani, 2019)	Merkle search tree	Authenticated ordered reconciliation
DataHub (Bhardwaj et al., 2015)	Version graph	Collaborative dataset motivation
Decibel (Maddox et al., 2016)	Relational layouts	Native database branching
OrpheusDB (Huang et al., 2017)	Partitioned versions	Storage/retrieval optimization
Delta Lake (Armbrust et al., 2020)	Log/checkpoints	ACID time travel at data-lake scale
Noms (Attic Labs, n.d.)	Merkle DAG	Structured content addressing
ForkBase (Wang et al., 2018)	Forkable CAS	Fork and deduplication semantics
DoltHub [2,3]	Prolly Trees	Mature production comparator

TABLE I. Related work and the scope of Revon.

2.4 Design Implications and Research Positioning

Versioning systems face three recurring costs: retaining history, creating a version, and later reconstructing or comparing versions. Snapshots simplify retrieval by copying unchanged records. Logs defer work until replay or checkpointing. Path copying creates only changed paths and shares the rest (Driscoll et al., 1989), (Bhattacharjee et al., 2015). Decibel and OrpheusDB show that physical layout affects both version creation and retrieval (Maddox et al., 2016), (Huang et al., 2017); graph-based analyses make the same storage/retrieval trade-off explicit (Guo et al., 2024). Reproducible-research work addresses a separate problem: preserving builds and artifacts for later inspection does not show that one storage algorithm is faster (Akhlaghi et al., 2021). For that reason, Revon reports import, commit, checkout, diff, and storage measurements separately.

Different research areas also give “change” different meanings. Differential dataflow incrementally recomputes query results as timestamped inputs change (McSherry et al., 2013). Data-provenance research asks why and where output facts were derived (Buneman et al., 2001), and DSLog stores and queries fine-grained array lineage (Zhao and Krishnan, 2024). Explain-Da-V uses data transformations to explain semantic dataset changes (Shraga and

Miller, 2023). Baseline uses operation-based differencing that can account for refactorings and schema changes (Edwards and Petricek, 2025). Revon reports a more limited result: which keys were added, removed, or changed in value between two states. It does not infer causes, transformation lineage, query impact, or schema intent. These systems offer useful context, but their outputs and workloads cannot be compared directly with Revon’s key/value diff timings.

Revon’s contribution is a controlled comparison of two correct diff paths over the same durable history, selected using properties of the requested interval. Each Revon-H commit stores both a Merkle root and an addressed changeset, so the choice of diff path does not change version semantics. We hypothesize that log aggregation suits short, sparse intervals and that hash pruning may help after longer operation sequences. The measured crossover depends on the workload and configuration. Work such as Git4Data also shows that database-native version control remains an active area (Gou et al., 2026), which narrows our claim: Revon evaluates adaptive differencing for a fixed-depth hash trie; it is not a general-purpose data-versioning platform. Snapshot and Log-only remain representation baselines. The public-record extension covers one release with generated updates. It does not establish performance across diverse datasets,

independent hardware, concurrent workloads, or production systems.

3 REVON DESIGN

Revon-H architecture

Service interface, versioning core and persistence

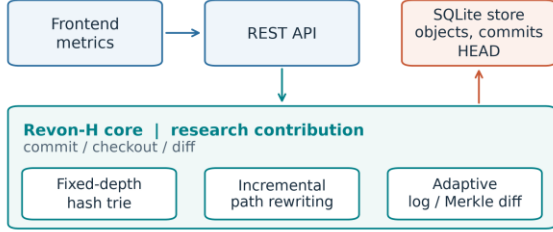


Fig. 1. Revon architecture and research scope.

3.1 Addressed Object Model

Revon serializes trie nodes and changesets as canonical JSON and names each object by its SHA-256 digest. A commit digest covers the root, parent, changeset, message, and timestamp. Labels such as v1 and v2 are readable aliases only; they do not affect object identity. A durable HEAD reference identifies the current commit.

3.2 Fixed-Depth Merkle Hash Trie

The main experiment uses $b=8$ and $d=4$. At each level, Revon reads three bits from SHA-256(key), so twelve routing bits select among 4,096 possible leaf buckets. Each bucket retains the full key, allowing Revon to distinguish keys with the same routing prefix. With uniform hashes, expected bucket occupancy is N/b^d . We evaluate other b, d pairs separately rather than treating this default as optimal.

3.3 Incremental Hashing and Commit

Revon canonicalizes each mutation batch, routes the keys, and groups updates that share trie prefixes. It rewrites each affected leaf once, then rebuilds only the ancestor paths that contain changes. Unchanged child hashes are reused in the new internal nodes, which preserves structural sharing. A single SQLite transaction writes the

new objects, changeset, commit metadata, version alias, measurements, and updated HEAD. On reopen, Revon verifies the stored payload hashes and references.

Incremental commit and hybrid diff

Update changed paths; reuse unchanged hashes

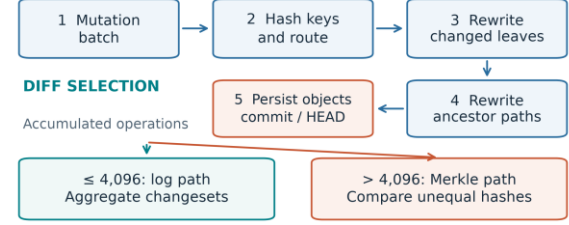


Fig. 2. Commit processing and adaptive diff selection.

3.4 Hybrid Diff

The log path walks the commit ancestry and aggregates key operations. The Merkle path compares roots recursively, skips subtrees with matching hashes, and examines full leaf entries only under changed paths. Revon-H counts the operations in the requested interval and uses the log path at or below 4,096 operations; above that threshold, it uses Merkle comparison. Revon-M always uses the Merkle path and serves as the ablation.

4 EXPERIMENTAL METHODOLOGY

4.1 Variants and Workloads

We compare five systems on the same logical states and mutation batches: Snapshot stores a complete copy per version; Log-only stores an initial state and operation log; Revon-M forces Merkle diff; Revon-H selects its diff path adaptively; and Dolt 2.3.1 uses a keyed SQL table and native JSON diff. We measure Dolt's initial import in two ways: batched SQL INSERT and CSV input through dolt table import -r. The CSV workflow includes serialization in its import time. Both import variants use the same Dolt engine for later operations, and we report their import results separately.

Scenario	Rows	Commits	Changed keys	Update locality
Small sparse	1,000	10	10	spread
Medium sparse	10,000	10	10	spread
Medium dense	10,000	10	1,000	spread
Large sparse	100,000	10	100	spread
Application-key local	100,000	10	100	lexically adjacent keys
Range local	100,000	10	100	contiguous sorted-key intervals
Repeated key	100,000	10	100	repeated small key set
Hash-route local	100,000	10	100	shared 9-bit SHA-256 prefix

TABLE II. Workloads used in the final evaluation.

4.2 Measurement Contract

We time initial import through the first durable state. For each trial, incremental-commit time is the median across ten commits. Diff and checkout timers include construction of a common output: diff returns sorted changed keys with old and new SHA-256 value hashes as canonical JSON, while checkout returns the full sorted key/value state as canonical UTF-8 JSON. Dolt’s diff time includes JSON parsing and normalization. We check both outputs against the workload oracle. Dolt’s SQL-path commit time includes SQL mutations, add, and commit; we retain commit hashes for historical checkout and create no version tags. Timed operations run without tracemalloc. During each isolated primary trial, an external parent process samples process-tree RSS every 10 ms, including Dolt child processes; sampling happens outside the timed operations. A separate Windows telemetry run records RSS, CPU, read/write bytes, and serial commit throughput in 405 rows (315 measured). Every row passed correctness checks and the schema-2 audit. These counters include setup and correctness work, not just timed operations. The WSL2 replication has incomplete I/O coverage and ran on the same physical host. Work-examined units differ by system and are not normalized.

4.3 Repetition, Calibration, and Integrity

The primary matrix uses two warm-ups and seven measured trials per configuration, for 120 warm-ups and 420 measured runs across calibration and evaluation. Of these, 280 measured runs belong to the five-system evaluation, including 56 SQL-path Dolt runs; another 56 measured runs use Dolt bulk import. We counterbalance and record model order, and run each system/trial in a clean worker. Each scenario uses

one deterministic workload, which the internal audit regenerates and checks by digest. The primary run used Windows 11 build 10.0.26200-SP0 and Python 3.14.3 (run b7a4ec7b610742d9b827f43fd5092fe2). For each scenario and comparison, we divide latency measurements from matching trial numbers and bootstrap the seven paired ratios 20,000 times to obtain a percentile 95% interval. These intervals describe variation across runs on this fixed workload and host; they do not measure variation across datasets or machines. We label intervals that include parity as inconclusive. The 5% practical margin is an analyst-selected label, not an external standard.

Threshold calibration

Separate workloads; medians and interquartile ranges

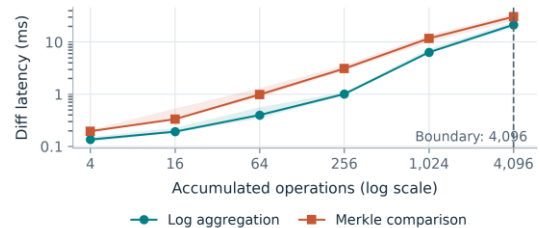


Fig. 3. Threshold calibration; 4,096 is the largest sampled point favoring the log path.

We fixed the 4,096-operation threshold for this machine; it is not a universal crossover. All 540 primary executions completed and passed the state and diff checks. The audit retained all 127 Tukey outlier candidates rather than removing them. A separate sensitivity run varies the seed, payload size, history length, and update pattern; its results are reported apart from the primary tables.

4.4 Public Dataset and Scalability Evaluation

We extend the synthetic evaluation with January 2024 yellow-taxi records from the New York

City Taxi and Limousine Commission (NYC TLC) (NYC TLC, 2024). The checksum-pinned Parquet file contains 2,964,624 rows; a dispersed affine permutation selects nested subsets of 10,000, 100,000 and 1,000,000 rows using seed 20261001. We retain eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Keys combine the release, file-checksum prefix and physical row ordinal, and therefore identify rows within this frozen file rather than trips across releases. Compact sorted-key UTF-8 JSON preserves values; nulls remain null. The million-row subset has 47,277 missing passenger counts and no serialization failures. Identical field values remain separate rows. The affine sample is not a uniformly random permutation.

Monthly trip files contain different trips, so we generate synthetic correction histories over these public records. Each commit samples distinct keys with a fixed seed, increases fare and total by 0.01, and preserves the other fields; keys may repeat across commits. Table III specifies the separate scale, history and density sweeps. All systems receive identical states and histories. We retain $b=8$, $d=4$ and the 4,096-operation selector. Diff compares version 1 with version $H+1$. Import is separate from incremental version creation; each trial reports the median of H commit intervals. Canonical output materialization is included in diff and checkout. Every trial checks both endpoint states and semantic diff; the first measured trial for each system/configuration checks every historical version outside timing. Directory footprint is measured before close and compaction. These are whole-system workflow measurements.

The public-data run used Python 3.12.14 on the same Windows host, with 15.64 GiB visible RAM and 4.14 GiB available at launch. Available memory was not held constant. Each configuration scheduled 2 warm-ups and 7 measured trials. All configurations have 7 successful measured trials. Clean workers and counterbalanced order follow the existing protocol; pilot measurements are excluded. Each trial has a

900-second wall-clock limit including setup and verification. We report medians and linearly interpolated interquartile ranges. 324 of 325 retained executions completed successfully. The source checksum, workload digests, raw trials, environment metadata, verification hashes and reproduction commands accompany the new evidence; prior bundles are preserved.

TABLE III. Public TLC dataset (NYC TLC, 2024), January 2024; all histories are generated corrections.

Parameter	Dataset and sweep specification
Dataset	NYC TLC yellow taxi, January 2024 (NYC TLC, 2024)
Records	2,964,624 source rows; 1,000,000 retained; nested subsets of 10k, 100k and 1M
Fields	Vendor; pickup/drop-off times; passenger count; distance; pickup/drop-off locations; payment type; fare; tip; total
Key	Month + source-checksum prefix + physical row ordinal; identity within the frozen file
Histories	Generated fare/total corrections; 10 or 50 incremental commits; no observed release histories
Scale	$N=10k/100k/1M$; $H=10$; $C=10/100/1,000$ per commit (0.1%)
History	$N=100k$; $H=10/50$; $C=100$ per commit (0.1%)
Density	$N=100k$; $H=10$; $C=10/100/1,000$ per commit (0.01%/0.1%/1%)
Sampling	Seed 20261001; nested prefixes of a dispersed affine permutation; not a uniform random permutation

5 RESULTS

5.1 Final Diff Performance

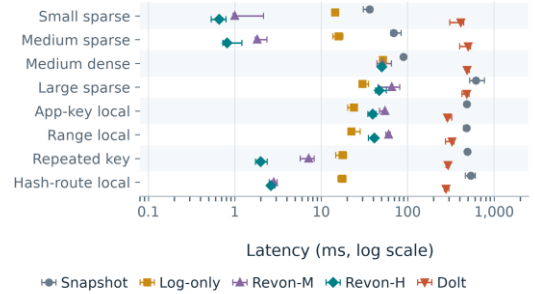


Fig. 4. Diff latency across eight update scenarios; lower values are better.

Scenario	Diff path	Revon-H (ms)	Dolt (ms)	SQL	Dolt/Revon-H ratio (95% CI)
Small sparse	log	0.659	411.69		625.1 (537.1-658.7)
Medium sparse	log	0.814	503.31		482.5 (393.4-652.0)
Medium dense	merkle	50.391	488.42		10.3 (9.0-10.8)
Large sparse	log	46.834	483.08		10.8 (8.2-12.4)
Application-key local	log	39.361	289.87		7.1 (6.8-9.8)
Range local	log	41.359	325.04		7.8 (6.8-11.2)
Repeated key	log	1.992	293.72		141.6 (109.5-173.2)
Hash-route local	log	2.620	278.22		102.4 (97.5-104.0)

TABLE IV. Revon-H and SQL-path Dolt diff latency; the final column gives the median paired ratio and 95% bootstrap interval.

In all eight scenarios, the paired Dolt/Revon-H diff ratio exceeded one and its 95% interval excluded parity. Revon-H therefore had lower diff latency in the workflows we tested. Table IV gives the median of the seven same-trial ratios and the bootstrap interval. The comparison is between Revon’s in-process API and Dolt’s CLI workflow, which includes Dolt CLI and SQL costs; these ratios are not engine-only speedups. The lexically adjacent application-key workload is distinct from the range-local, repeated-key, and hash-route-local workloads.

5.2 Hybrid Ablation

Three of the eight primary Revon-M/Revon-H diff intervals included parity, so those comparisons were inconclusive. The separate robustness matrix used 10,000 rows, three seeds, 32- and 128-byte values, histories of 10 and 50 commits, and spread, range-local, and repeated-key updates. A Revon-M/Revon-H ratio above one means Revon-M was slower. On Windows, the ratios for the baseline, 128-byte values, range-local updates, and repeated-key updates were 1.58 [1.23, 1.87], 1.62 [1.38, 1.72], 1.64 [1.38, 1.77], and 2.24 [2.02, 2.38]. The 50-commit interval crossed parity. WSL2 showed the same direction for the other four factors, while the 50-commit interval again overlapped parity. This three-seed sensitivity study on one physical host offers more direct evidence about adaptive selection than the cross-system ratios, but it does not establish general superiority.

5.3 Commit, Checkout, and Storage

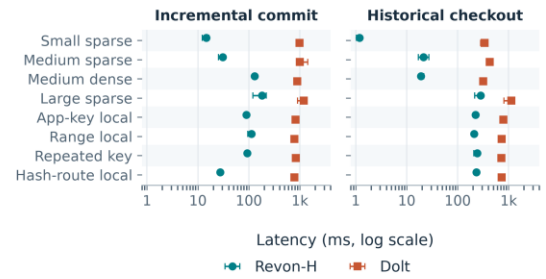


Fig. 5. Operational latency for Revon-H and Dolt; lower values are better.

Across all eight scenarios, the paired Dolt/Revon-H intervals excluded parity for initial import, incremental commit, diff, and historical checkout. Full ratios and intervals appear in `paired_ratio_uncertainty.csv`. Median initial-import times across scenario medians were 1,814.83 ms for batched SQL INSERT, 1,701.25 ms for Dolt bulk import, and 1,131.24 ms for Revon-H. In the paired SQL-versus-bulk-import analysis, four scenario intervals favored bulk import and four included parity; the results do not show a general bulk-import advantage. These are workflow measurements, not evidence of engine-level or production-system superiority.

Repository directory size measures the footprint of the full workflow, not storage normalized to the algorithms alone. Across the eight scenarios, Revon-H’s repository footprint was 1.99–6.90 times Dolt’s. After dividing each directory size by the final UTF-8 key/value payload, the median Revon-H/Dolt ratio was 3.82; the corresponding ratio for the residual was 6.82. That residual is not pure metadata: it also includes serialization, indexes, other metadata, and compression effects. After compaction, mean sampled footprints were 22.08 MiB for Revon-M, 22.08 MiB for Revon-H, 2.17 MiB for Dolt SQL, and 2.19 MiB for Dolt bulk import. We compacted Dolt offline with ``dolt gc`` and the

Revon SQLite repositories with 'VACUUM'; compaction time is excluded from latency and RSS. These are sampled outcomes, not a pure metadata comparison or a universal steady-state result.

5.4 Trie-Parameter Sensitivity

Fixed-trie geometry sensitivity

10,000 rows; forced-Merkle diff; seven runs per configuration

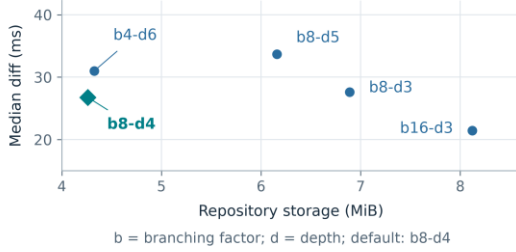


Fig. 6. Fixed-trie sensitivity; points show median diff latency, and the axis extends beyond the largest measured 75th percentile.

The corrected fixed-trie sensitivity run completed 45 executions: 10 warm-ups and 35 measured runs. The b16-d3 configuration had the lowest median diff time (21.46 ms), but used 8.12 MiB. The b8-d4 configuration took 26.74 ms and used the least storage (4.26 MiB). The over-deep b8-d5 configuration took 33.68 ms to diff and used 44.5% more storage than b8-d4. Thus, b8-d4 balances latency and storage in this sample; it is not a universal optimum.

5.5 Public Dataset Scale History and Update Density

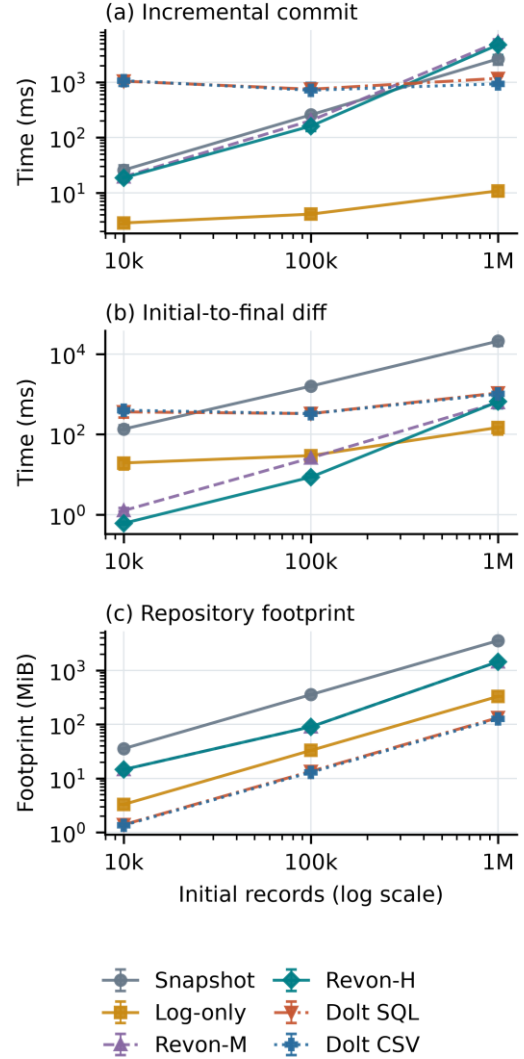


Fig. 7. Public-record scalability with ten generated commits and 0.1% updates per commit; medians and interquartile ranges (seven successful measured trials per configuration). Both axes use logarithmic scales. Revon-H uses log diff at 10k/100k and Merkle diff at 1M.

At one million records, Revon-H's median incremental commit was 4764.26 [3086.15, 5159.49] ms and its initial-to-final diff was 651.01 [448.66, 693.13] ms. Its median repository footprint was 1446.69 MiB. Both Revon variants selected Merkle differencing for the 10,000 accumulated operations at one million records; this configuration does not test a log-path benefit. Revon-H's incremental-commit median exceeded those of Snapshot, Log-only, Dolt SQL, Dolt CSV at this scale.

Initial import for Revon-H took 39.60 [27.54, 41.41] s, compared with 29.02 [20.93, 37.70] s for Dolt SQL and 24.94 [20.11, 27.29] s for Dolt CSV import. These initial loads are separate from incremental commits. The ratio of median Revon-H/Dolt SQL repository footprints was 10.90; this includes serialization, indexes and compression effects and is not a pure metadata comparison.

Brackets give the interquartile range across measured trial summaries, not confidence intervals or variation across datasets. Full import, checkout, storage and failure records remain in the evidence bundle.

Failed executions are retained and excluded from performance summaries. One interrupted million-record Revon-M attempt was followed by a fresh recovery trial after the matrix, under identical frozen source and workload. Seven successful measured trials enter each summary; the failed attempt remains in the original bundle. The recovery was outside the counterbalanced block and host memory conditions were not held constant.

TABLE V. Public-data history and density results at 100,000 records; median commit and diff times in ms and footprint in MiB. Full interquartile ranges are retained in summary.csv.

H / density	System	Commit	Diff	MiB
10 / 0.1%	Snapshot	257.41	1584.12	354.91
10 / 0.1%	Log-only	4.11	29.62	32.93
10 / 0.1%	Revon-M	202.78	25.95	90.03
10 / 0.1%	Revon-H	161.09	8.65	90.04
10 / 0.1%	Dolt SQL	749.08	329.10	13.50
10 / 0.1%	Dolt CSV	716.78	327.17	12.95
50 / 0.1%	Snapshot	219.28	1377.48	1645.60
50 / 0.1%	Log-only	3.26	52.11	35.58
50 / 0.1%	Revon-M	135.36	83.68	137.24
50 / 0.1%	Revon-H	141.22	91.79	137.24
50 / 0.1%	Dolt SQL	882.14	590.91	25.88
50 / 0.1%	Dolt CSV	873.70	525.45	25.33
10 / 0.01%	Snapshot	236.94	1257.26	354.90
10 / 0.01%	Log-only	3.00	21.38	32.33
10 / 0.01%	Revon-M	26.57	2.78	79.68
10 / 0.01%	Revon-H	28.56	0.78	79.68
10 / 0.01%	Dolt SQL	1001.47	374.25	10.85
10 / 0.01%	Dolt CSV	1015.52	367.27	10.30
10 / 1%	Snapshot	265.73	1738.57	354.97
10 / 1%	Log-only	11.04	146.20	38.90
10 / 1%	Revon-M	752.02	164.79	177.68
10 / 1%	Revon-H	679.38	166.75	177.66
10 / 1%	Dolt SQL	943.05	740.59	25.31
10 / 1%	Dolt CSV	976.99	702.91	24.75

At 100,000 records, the 10-commit 0.1% case accumulates 1,000 operations, while the 50-commit case accumulates 5,000 and the 1% case accumulates 10,000. Revon-H therefore uses the log path for the sparse short history and the Merkle path for the latter cases. This sweep evaluates the frozen selector under public record contents; it does not establish a new threshold or universal advantage. The update-only histories differ from the earlier mixed synthetic workloads.

6 DISCUSSION

For RQ1, the results show workload-dependent latency and storage trade-offs across the synthetic update patterns we tested. In a separate three-seed study, Revon-H had lower diff times than the forced-Merkle ablation for four factors on both Windows and WSL2. The 50-commit comparison remained inconclusive on both. Lexically adjacent application keys, sorted-range updates, repeated-key updates, and hash-

route locality represent different patterns. We did not test ordered range queries, concurrent readers or writers, or production throughput.

For RQ2, the paired Revon-M/Revon-H ablation provides the clearest comparison: both variants use the same durable trie, and differ in diff-path selection. Revon-H showed advantages for the tested short and sparse histories. The 50-commit case, which crossed the fixed operation threshold, showed no resolved difference. These results do not establish a universal advantage for the selector.

For RQ3, the results rule out one trie geometry as best for every workload. Shallow routing reduces internal traversal but leaves larger buckets and higher storage use. Deeper routing reduces bucket occupancy but adds internal-node work. The suitable configuration depends on whether a workload prioritizes latency, storage, or limiting leaf scans.

The Dolt ratios reflect the complete systems and interfaces we measured. Dolt handles SQL parsing, schema management, version-control operations, and production durability, which the Revon prototype does not. Its Prolly Trees also support ordered queries, branches, merges, and remote workflows. Because our timings include each system’s declared interface, they do not isolate the tree algorithms. We therefore cannot conclude that a fixed trie is inherently faster than a Prolly Tree.

The public-data sweeps extend the evidence to real record contents and larger states, but the histories remain generated. At fixed depth, increasing N increases expected leaf occupancy, so rewriting touched leaves can become more expensive even with an unchanged update percentage. The scale sweep also crosses the frozen selector threshold: Revon-H uses log diff at 10k/100k records and Merkle diff at 1M. Its diff curve therefore combines larger states with a strategy change. Longer histories likewise accumulate operations and can move Revon-H to the Merkle path. Figure 7 and Table V describe this host and record mapping; they do not isolate a tree algorithm, test relational queries, or measure naturally occurring corrections.

7 LIMITATIONS AND THREATS TO VALIDITY

- Runtime and interface: Revon runs in-process in Python; Dolt is a mature Go executable invoked through CLI and SQL interfaces. This difference affects the measured workflow.

- Feature set: Revon supports one writer and a linear history. It does not support SQL, branches, merges, remotes, schema evolution, or production concurrency.

- Dataset scope: The earlier synthetic study reaches 100,000 rows; the new public-record evaluation completes configurations up to 1,000,000 rows from one TLC monthly file with eleven retained fields. All public-data updates are generated, the scale sample uses one seed and an affine permutation, and key identities are valid only within the frozen release. We did not evaluate observed histories, multiple public datasets, multi-table workloads, relational queries or scale beyond the completed configurations.

- Environment and replication: The primary run used one Windows 11 host. The WSL2 Ubuntu sensitivity run used the same physical computer, not an independent Linux machine. No external researcher or clean clone has reproduced the timings. The internal audit checks evidence consistency; it does not reproduce wall-clock performance.

- Calibration: The log path was faster at the largest sampled operation count that favored it, 4,096. This result does not establish a universal crossover.

- Memory: We measured median whole-worker process-tree RSS in eight primary scenarios for Snapshot, Log-only, Revon-M, Revon-H, and SQL-path Dolt. These measurements include shared workload/oracle state and adapter processes, so they are not product-only memory figures. The parent process sampled every 10 ms outside timed operations. It may miss shorter peaks, and monitoring may add small scheduling pressure. Dolt bulk-import measurements are reported separately from SQL-path Dolt. Public-data trials also include workload/oracle memory. The host had limited available RAM at launch; memory pressure and paging costs were not isolated from the workflow, so

the measured scaling should not be read as an algorithm-only complexity result.

- Work units: Keys, log operations, trie-node pairs, and Dolt’s internal work are different quantities. We do not combine them into a normalized work measure.

- Uncertainty: We retained every Tukey outlier candidate. The primary seven-trial ratio intervals are paired bootstrap intervals for each fixed scenario. The additional seed-cluster bootstrap uses only three seeds. Neither analysis estimates performance across a population of datasets or machines.

8 FUTURE WORK

A future version could select its structure from observed workload statistics. Key count, prefix skew, mutation density, and leaf occupancy may help choose among the tested trie geometries instead of fixing b and d for every repository. The diff selector could also recalibrate over time, using history distance, changed-prefix density, and observed I/O alongside operation count.

Revon’s storage could be reduced through binary node records, separate value chunks, packed objects, compression, reachability checks, and garbage collection. An ordered Merkle index or content-defined chunking could add range scans and history-independent boundaries while retaining the fixed trie as a controlled baseline.

Moving beyond a linear prototype will require named branches, merge commits, conflict reporting, schema-aware records, concurrent readers and writers, ordered range scans, and remote synchronization. Evaluation also needs additional public datasets, observed update histories, larger feasible states, and replications by independent researchers on separate hardware. WSL2 I/O counters remain incomplete, and the Windows sensitivity counters cover the workflow rather than isolating individual operation costs.

9 CONCLUSION

Revon demonstrates one way to version structured key/value data using immutable objects, incremental Merkle updates, SQLite persistence, historical checkout, and adaptive differencing. Paired intervals and a three-seed sensitivity study show workload-specific differences be-

tween Revon-H and the forced-Merkle ablation; the long-history result remains inconclusive. The public-record extension reaches 1,000,000 records from one TLC release using generated corrections. It does not establish performance across datasets, observed histories, independent hardware, concurrent workloads or ordered range queries. The evidence describes a prototype, not a production-system replacement.

ACKNOWLEDGMENT

We thank the open-source Dolt project and the researchers whose work on dataset versioning informed our experimental design.

REFERENCES

- Scott Chacon and Ben Straub. 2014. Git Internals: Git Objects. In *Pro Git*, second edition. Available at <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
- DoltHub. 2024. Dolt Storage Engine: Block Store. *Dolt Documentation*. Available at <https://www.dolthub.com/docs/architecture/storage-engine/block-store/>.
- Tim Sehn. 2024. Dolt’s Storage Engine. *DoltHub Blog*, February 29. Available at <https://www.dolthub.com/blog/2024-02-29-storage-engine/>.
- Ralph C. Merkle. 1988. A digital signature based on a conventional encryption function. In *Advances in Cryptology: CRYPTO ’87, Lecture Notes in Computer Science 293*, 369–378. Springer. https://doi.org/10.1007/3-540-48184-2_32.
- James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. 1989. Making data structures persistent. *Journal of Computer and System Sciences* 38, 1 (1989), 86–124. [https://doi.org/10.1016/0022-0000\(89\)90034-2](https://doi.org/10.1016/0022-0000(89)90034-2).
- Juan Benet. 2014. IPFS: Content Addressed, Versioned, P2P File System. *arXiv:1407.3561*.
- Alex Auvolat and François Taïani. 2019. Merkle Search Trees: Efficient State-Based CRDTs in Open Networks. In *Proceedings of the 38th IEEE International Symposium on Reliable Distributed Systems*, 221–230. <https://doi.org/10.1109/SRDS47363.2019.00032>.
- Athicha Muthitacharoen, Benjie Chen, and David Mazières. 2001. A low-bandwidth network file system. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles*, 174–187. <https://doi.org/10.1145/502034.502052>.
- Anant Bhardwaj et al. 2015. DataHub: Collaborative Data Science and Dataset Version Management

- at Scale. In *Proceedings of the 7th Biennial Conference on Innovative Data Systems Research*.
- Michael Maddox et al. 2016. Decibel: The Relational Dataset Branching System. *Proceedings of the VLDB Endowment* 9, 9 (2016), 624–635. <https://doi.org/10.14778/2947618.2947619>.
- Silu Huang, Liqi Xu, Jialin Liu, Aaron J. Elmore, and Aditya G. Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. *Proceedings of the VLDB Endowment* 10, 10 (2017), 1130–1141. <https://doi.org/10.14778/3115404.3115417>.
- Souvik Bhattacharjee, Amit Chavan, Silu Huang, Amol Deshpande, and Aditya Parameswaran. 2015. Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1346–1357. <https://doi.org/10.14778/2824032.2824035>.
- Michael Armbrust et al. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3411–3424. <https://doi.org/10.14778/3415478.3415560>.
- Attic Labs. n.d. Noms Technical Overview. *GitHub*. Available at <https://github.com/attic-labs/noms/blob/master/doc/intro.md>.
- Sheng Wang et al. 2018. ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications. *Proceedings of the VLDB Endowment* 11, 10 (2018), 1137–1150. <https://doi.org/10.14778/3231751.3231762>.
- Andy Twigg, Andrew Bye, Grzegorz Miłos, Tim Moreton, John Wilkes, and Tom Wilkie. 2011. Stratified B-trees and versioning dictionaries. In *Proceedings of the 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*. Open version: <https://arxiv.org/abs/1103.4282>.
- Adam Seering, Philippe Cudré-Mauroux, Samuel Madden, and Michael Stonebraker. 2012. Efficient Versioning for Scientific Array Databases. In *Proceedings of the 28th IEEE International Conference on Data Engineering (ICDE)*, 1013–1024. <https://doi.org/10.1109/ICDE.2012.102>. Public author-uploaded full text: https://www.researchgate.net/publication/241633210_Efficient_Versioning_for_Scientific_Array_Databases.
- Emad Soroush and Magdalena Balazinska. 2013. Time Travel in a Scientific Array Database. In *Proceedings of the 29th IEEE International Conference on Data Engineering (ICDE)*, 98–109. <https://doi.org/10.1109/ICDE.2013.6544817>. Open author copy: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.
- Gerth Stølting Brodal, Spyros Sioutas, Konstantinos Tsakalidis, and Kostas Tsichlas. 2020. Fully Persistent B-trees. *Theoretical Computer Science* 841 (2020), 10–26. <https://doi.org/10.1016/j.tcs.2020.06.027>. Open author copy: <https://cs.au.dk/~gerth/papers/tcs20.pdf>.
- Nazmus Saquib, Chandra Krintz, and Rich Wolski. 2023. Replicated Versioned Data Structures for Wide-Area Distributed Systems. *IEEE Transactions on Parallel and Distributed Systems* 34, 1 (2023), 207–224. <https://doi.org/10.1109/TPDS.2022.3217969>. Open author copy: <https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf>.
- Maximilian E. Schüle, Lukas Karnowski, Josef Schmeißer, Benedikt Kleiner, Alfons Kemper, and Thomas Neumann. 2019. Versioning in Main-Memory Database Systems: From MusaeusDB to TardisDB. In *Proceedings of the 31st International Conference on Scientific and Statistical Database Management (SSDBM)*, 169–180. <https://doi.org/10.1145/3335783.3335792>. Open author copy: <https://www-db.cs.tum.edu/~schuele/data/tardis.pdf>.
- Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and Where: A Characterization of Data Provenance. In *Database Theory: ICDT 2001*, Lecture Notes in Computer Science 1973, 316–330. https://doi.org/10.1007/3-540-44503-X_20. Open author copy: https://www.research.ed.ac.uk/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.
- Frank McSherry, Derek G. Murray, Rebecca Isaacs, and Michael Isard. 2013. Differential Dataflow. In *Proceedings of the 6th Biennial Conference on Innovative Data Systems Research (CIDR)*. Open proceedings paper: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.
- Mohammad Akhlaghi, Raúl Infante-Sainz, Boudewijn F. Roukema, Mohammadreza Khellat, David Valls-Gabaud, and Roberto Baena-Gallé. 2021. Toward Long-Term and Archivable Reproducibility. *Computing in Science & Engineering* 23, 3 (2021), 82–91. <https://doi.org/10.1109/MCSE.2021.3072860>. Open preprint: <https://arxiv.org/abs/2006.03018>.
- Roe Shraga and Renée J. Miller. 2023. Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V (Technical Report). arXiv:2301.13095. Preprint: <https://arxiv.org/abs/2301.13095>.
- Anxin Guo, Jingwei Li, Pattara Sukprasert, Samir Khuller, Amol Deshpande, and Koyel Mukherjee. 2024. To Store or Not to Store: A Graph Theoretical Approach for Dataset Versioning. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. <https://doi.org/10.1109/IPDPS57955.2024.00049>. Open preprint: <https://arxiv.org/abs/2402.11741>.
- Alba González-Cebrián, Michael Bradford, Adriana E. Chis, and Horacio González-Vélez. 2024. Standardised Versioning of Datasets: A FAIR-Compliant Proposal. *Scientific Data* 11 (2024), Ar-

title 358. <https://doi.org/10.1038/s41597-024-03153-y>. Open access: <https://www.nature.com/articles/s41597-024-03153-y>.

Jinjin Zhao and Sanjay Krishnan. 2024. Compression and In-Situ Query Processing for Fine-Grained Array Lineage. arXiv:2405.17701. Preprint: <https://arxiv.org/abs/2405.17701>.

Hongshen Gou, Zuyu Zhang, Yuze Sun, Peng Xu, Feng Tian, Long Wang, and Jianguo Wang. 2026. Git4Data: Database-Native Version Control for AI Agents. arXiv:2609.02106. Preprint: <https://arxiv.org/abs/2609.02106>.

Jonathan Edwards and Tomas Petricek. 2025. Baseline: Operation-Based Evolution and Versioning of Data. arXiv:2512.09762 (2025; updated 2026). Preprint: <https://arxiv.org/abs/2512.09762>.

New York City Taxi and Limousine Commission. 2024. TLC Trip Record Data: Yellow Taxi, January 2024. Public dataset. Retrieved October 1, 2026 from <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.

APPENDIX A. REPRODUCIBILITY AND DATA AVAILABILITY

The repository includes the benchmark harness, evidence audit, paired-ratio analysis, multi-seed robustness study, and manuscript validation tools. The corrected primary paper-profile bundle is `evidence/paper-corrected-issues13-17-counterbalanced-final-20260923`; its paired ratio intervals are in `paired_ratio_uncertainty.csv`. The

Windows and WSL2 sensitivity bundles are `evidence/paper-issues18-20-robustness-20260923` and `evidence/paper-issues18-20-linux-wsl-20260923`. Supplemental Windows telemetry is in `evidence/paper-issues14-windows-telemetry-20260924`, and trie sensitivity results are in `evidence/trie-sensitivity-issues8-20260923`. These bundles retain raw rows, manifests, audits, summaries, and seed-cluster intervals. They support checks of evidence consistency; they do not independently reproduce timing results. The project repository is <https://github.com/atharvasheersh/Revon>. Public dataset acquisition and preparation are documented in `docs/experiments/PUBLIC_DATASET_EVALUATION.md`; `experiments/public_dataset.py` and `experiments/public_dataset_study.py` regenerate the checksum-pinned workload and run its sweeps. The TLC source is cited in (NYC TLC, 2024). New bundles are `evidence/public-tlc-combined-20261002`. Raw and prepared trip records are downloaded locally and excluded from Git; retained timings, manifests, verification hashes and audited summaries are available with the code.